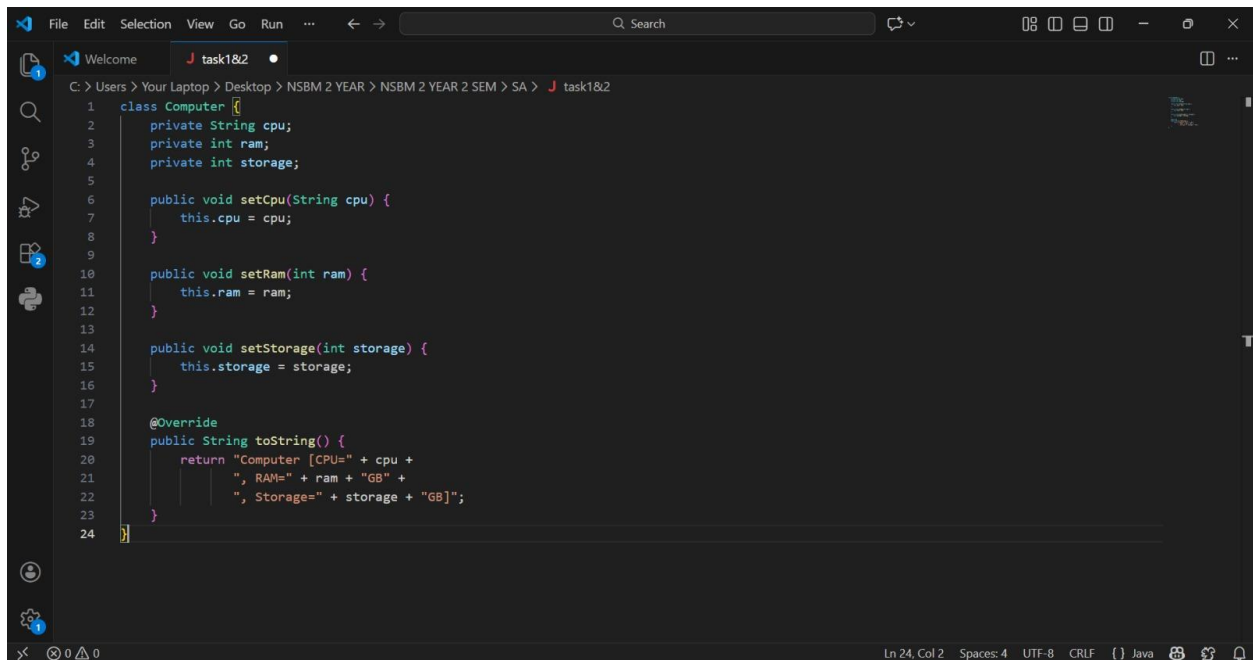
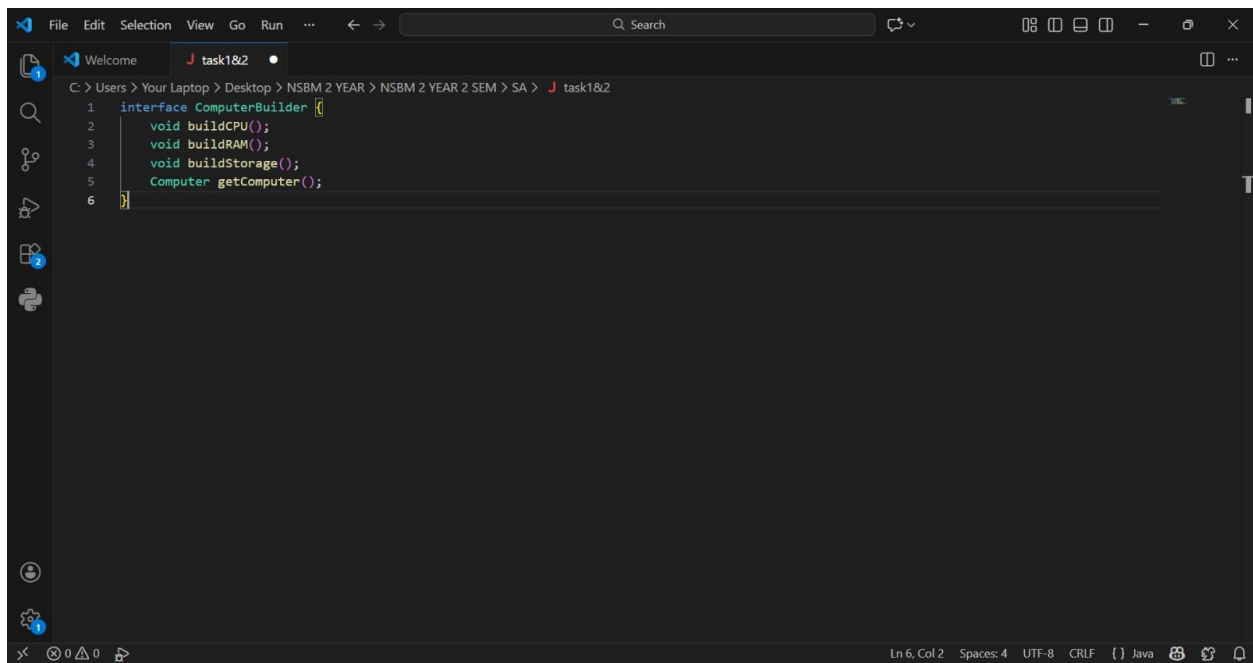


TASK 01



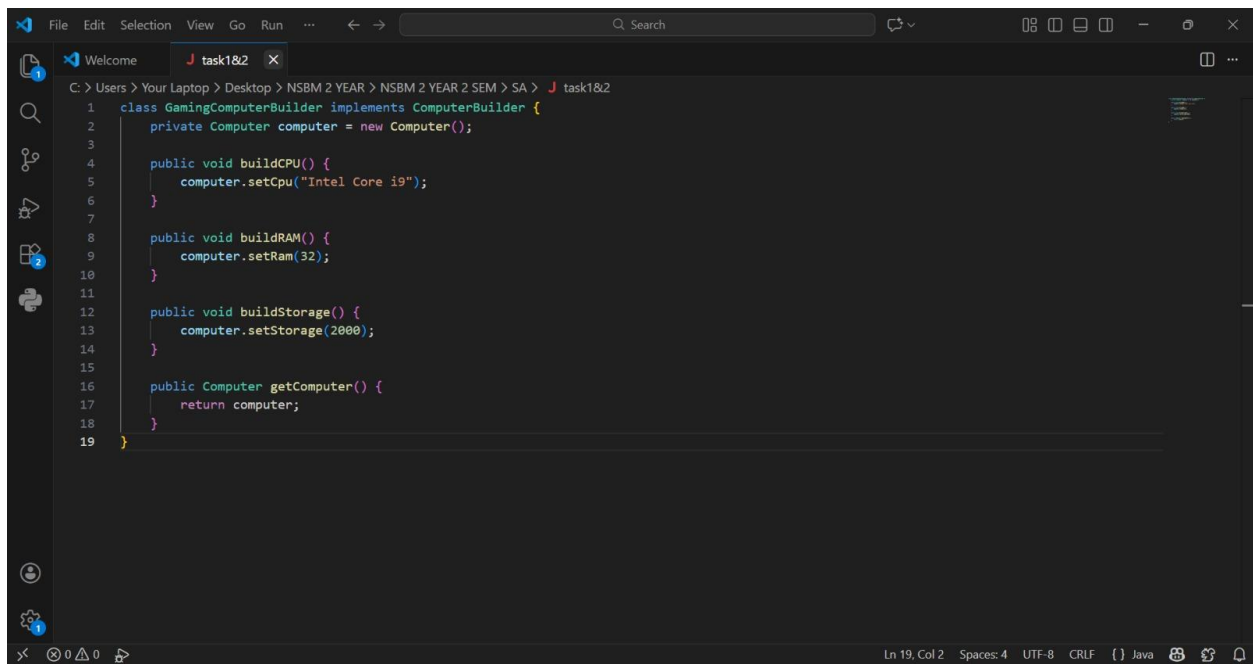
```
File Edit Selection View Go Run ... Search
Welcome J task1&2
C: > Users > Your Laptop > Desktop > NSBM 2 YEAR > NSBM 2 YEAR 2 SEM > SA > J task1&2
1 class Computer {
2     private String cpu;
3     private int ram;
4     private int storage;
5
6     public void setCpu(String cpu) {
7         this.cpu = cpu;
8     }
9
10    public void setRam(int ram) {
11        this.ram = ram;
12    }
13
14    public void setStorage(int storage) {
15        this.storage = storage;
16    }
17
18    @Override
19    public String toString() {
20        return "Computer [CPU=" + cpu +
21            ", RAM=" + ram + "GB" +
22            ", Storage=" + storage + "GB]";
23    }
24 }
```

Ln 24, Col 2 Spaces: 4 UTF-8 CRLF {} Java



```
File Edit Selection View Go Run ... Search
Welcome J task1&2
C: > Users > Your Laptop > Desktop > NSBM 2 YEAR > NSBM 2 YEAR 2 SEM > SA > J task1&2
1 interface ComputerBuilder {
2     void buildCPU();
3     void buildRAM();
4     void buildStorage();
5     Computer getComputer();
6 }
```

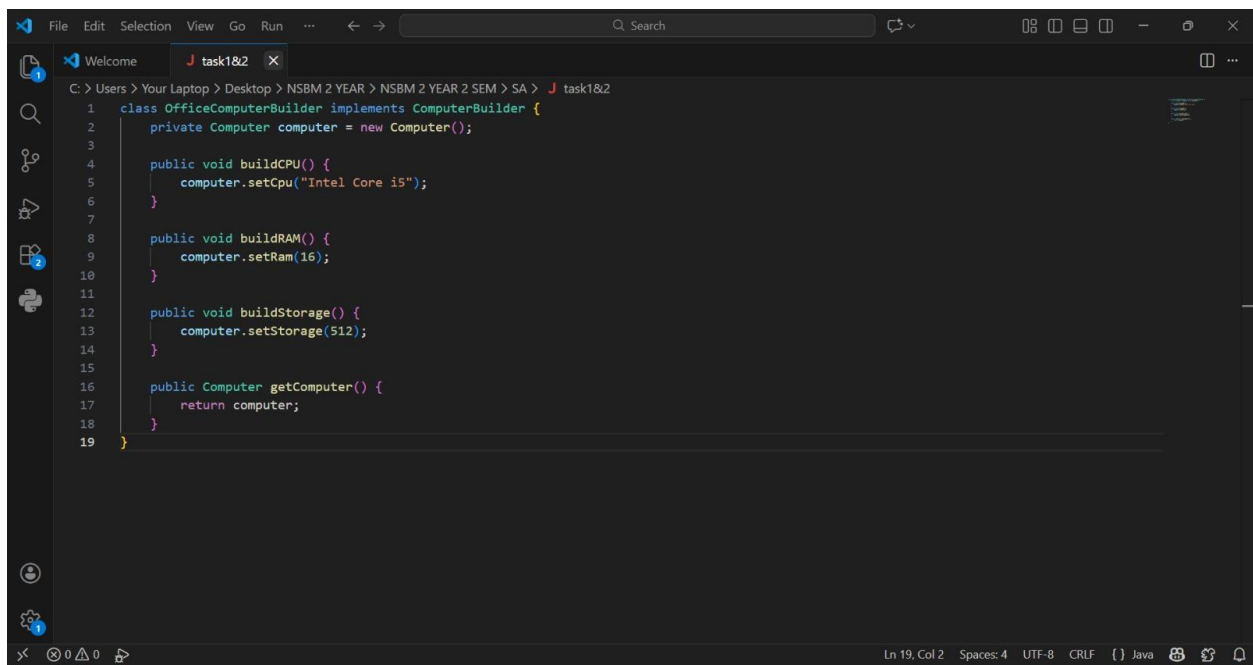
Ln 6, Col 2 Spaces: 4 UTF-8 CRLF {} Java



This screenshot shows an IDE window with a single tab titled 'task1&2'. The code is a Java class named 'GamingComputerBuilder' that implements the 'ComputerBuilder' interface. The class has a private 'computer' attribute of type 'Computer'. It includes methods for building the CPU, RAM, and storage, and a method to retrieve the built computer. The file path is 'C:\Users\Your Laptop\Desktop\NSBM 2 YEAR\NSBM 2 YEAR 2 SEM\SA\J task1&2'.

```
1 class GamingComputerBuilder implements ComputerBuilder {
2     private Computer computer = new Computer();
3
4     public void buildCPU() {
5         computer.setCpu("Intel Core i9");
6     }
7
8     public void buildRAM() {
9         computer.setRam(32);
10    }
11
12    public void buildStorage() {
13        computer.setStorage(2000);
14    }
15
16    public Computer getComputer() {
17        return computer;
18    }
19 }
```

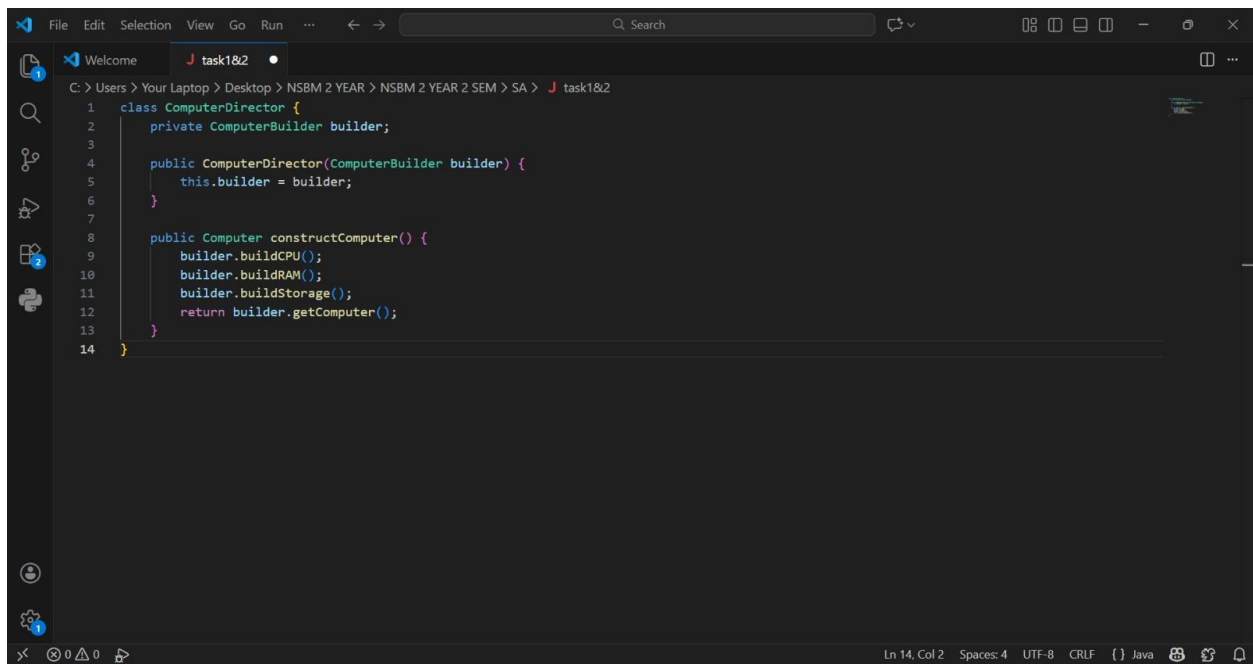
Ln 19, Col 2 Spaces: 4 UTF-8 CRLF {} Java



This screenshot shows an IDE window with a single tab titled 'task1&2'. The code is a Java class named 'OfficeComputerBuilder' that implements the 'ComputerBuilder' interface. The class has a private 'computer' attribute of type 'Computer'. It includes methods for building the CPU, RAM, and storage, and a method to retrieve the built computer. The file path is 'C:\Users\Your Laptop\Desktop\NSBM 2 YEAR\NSBM 2 YEAR 2 SEM\SA\J task1&2'.

```
1 class OfficeComputerBuilder implements ComputerBuilder {
2     private Computer computer = new Computer();
3
4     public void buildCPU() {
5         computer.setCpu("Intel Core i5");
6     }
7
8     public void buildRAM() {
9         computer.setRam(16);
10    }
11
12    public void buildStorage() {
13        computer.setStorage(512);
14    }
15
16    public Computer getComputer() {
17        return computer;
18    }
19 }
```

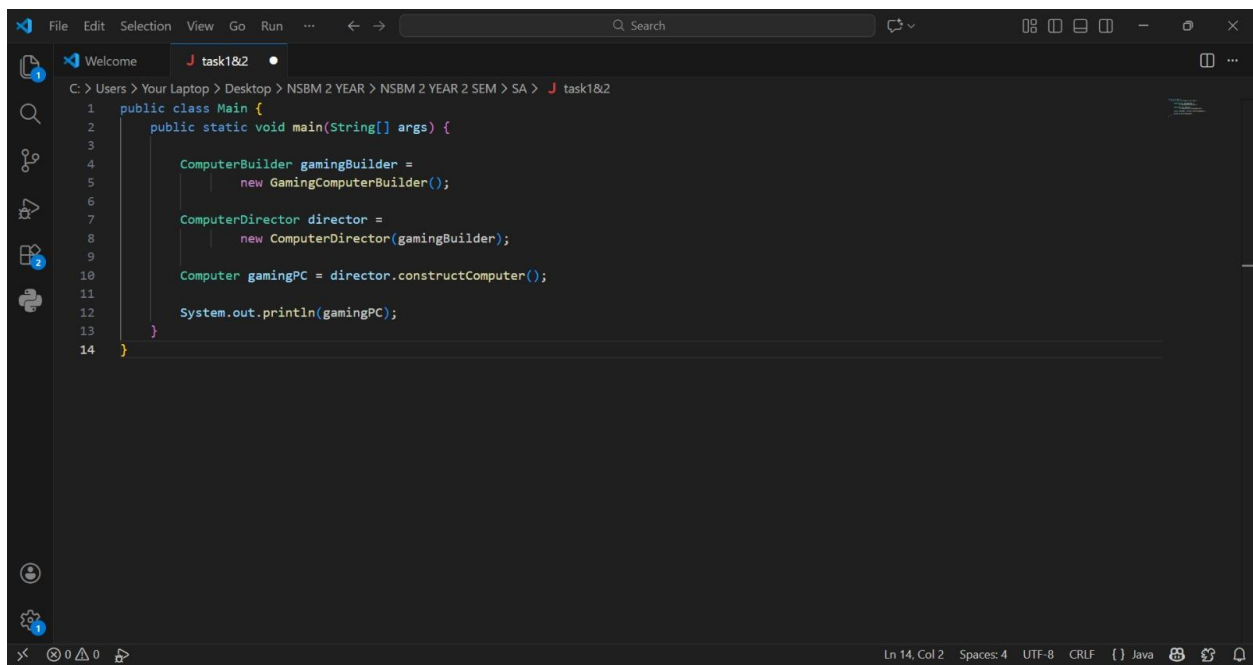
Ln 19, Col 2 Spaces: 4 UTF-8 CRLF {} Java



This screenshot shows an IDE window with a single tab titled "task1&2". The file path is "C:\Users\Your Laptop\Desktop\NSBM 2 YEAR\NSBM 2 YEAR 2 SEM\SA\task1&2". The code defines a `ComputerDirector` class with a private `ComputerBuilder` field, a constructor, and a `constructComputer` method that delegates the building process to the builder.

```
1 class ComputerDirector {
2     private ComputerBuilder builder;
3
4     public ComputerDirector(ComputerBuilder builder) {
5         this.builder = builder;
6     }
7
8     public Computer constructComputer() {
9         builder.buildCPU();
10        builder.buildRAM();
11        builder.buildStorage();
12        return builder.getComputer();
13    }
14 }
```

The status bar at the bottom indicates the cursor is at line 14, column 2, with 4 spaces, UTF-8 encoding, CRLF line endings, and the file is a Java source file.

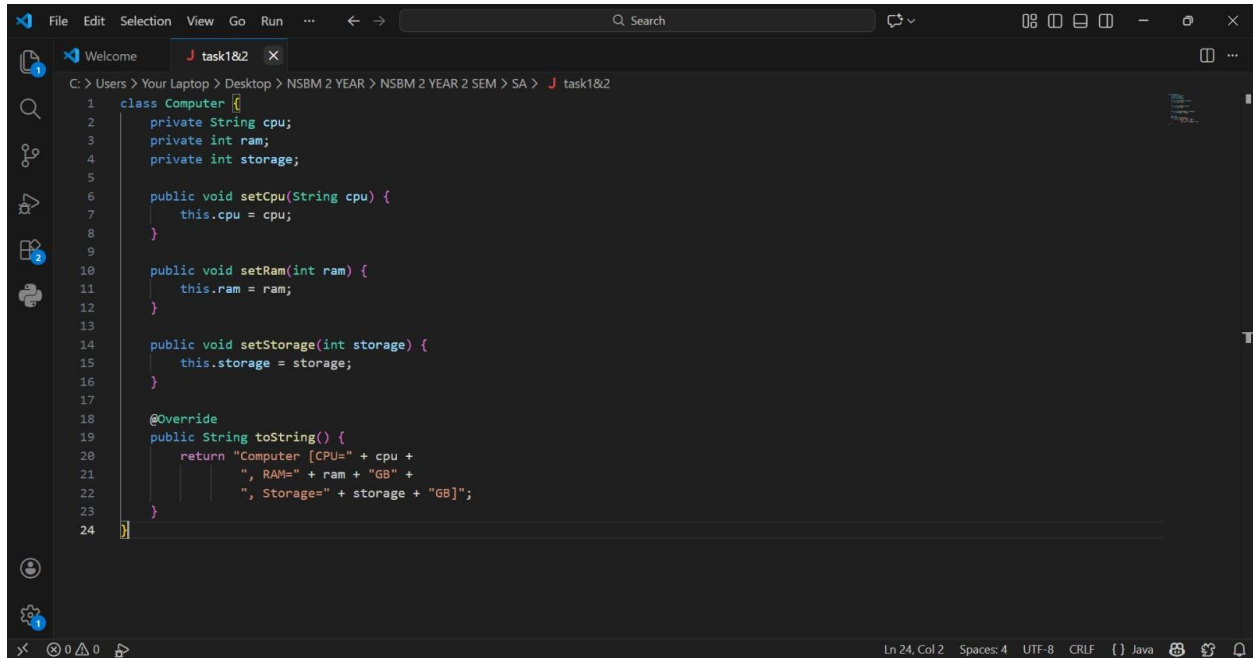


This screenshot shows the same IDE window with the `Main` class. The file path is "C:\Users\Your Laptop\Desktop\NSBM 2 YEAR\NSBM 2 YEAR 2 SEM\SA\task1&2". The `Main` class contains a `main` method that creates a `GamingComputerBuilder`, a `ComputerDirector` using it, and then uses the director to build and print a `gamingPC`.

```
1 public class Main {
2     public static void main(String[] args) {
3
4         ComputerBuilder gamingBuilder =
5             new GamingComputerBuilder();
6
7         ComputerDirector director =
8             new ComputerDirector(gamingBuilder);
9
10        Computer gamingPC = director.constructComputer();
11
12        System.out.println(gamingPC);
13    }
14 }
```

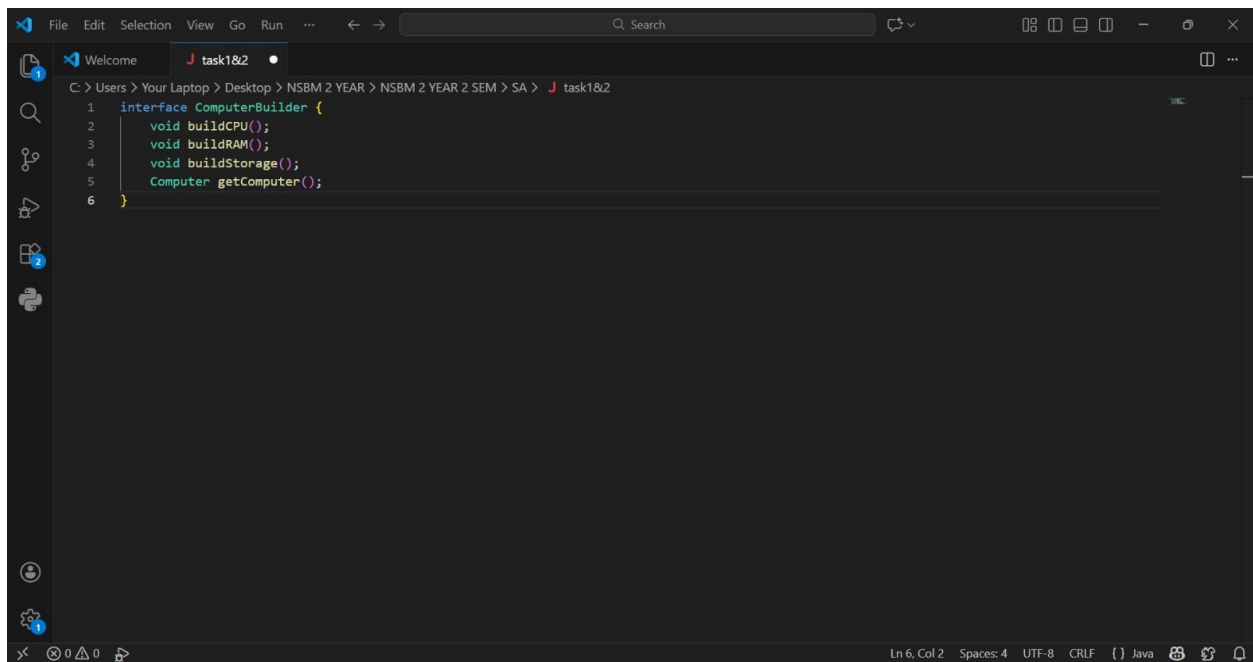
The status bar at the bottom indicates the cursor is at line 14, column 2, with 4 spaces, UTF-8 encoding, CRLF line endings, and the file is a Java source file.

TASK 02



```
File Edit Selection View Go Run ... Search
Welcome J task1&2
C: > Users > Your Laptop > Desktop > NSBM 2 YEAR > NSBM 2 YEAR 2 SEM > SA > J task1&2
1 class Computer {
2     private String cpu;
3     private int ram;
4     private int storage;
5
6     public void setCpu(String cpu) {
7         this.cpu = cpu;
8     }
9
10    public void setRam(int ram) {
11        this.ram = ram;
12    }
13
14    public void setStorage(int storage) {
15        this.storage = storage;
16    }
17
18    @Override
19    public String toString() {
20        return "Computer [CPU=" + cpu +
21            ", RAM=" + ram + "GB" +
22            ", Storage=" + storage + "GB]";
23    }
24 }
```

Ln 24, Col 2 Spaces: 4 UTF-8 CRLF {} Java



```
File Edit Selection View Go Run ... Search
Welcome J task1&2
C: > Users > Your Laptop > Desktop > NSBM 2 YEAR > NSBM 2 YEAR 2 SEM > SA > J task1&2
1 interface ComputerBuilder {
2     void buildCPU();
3     void buildRAM();
4     void buildStorage();
5     Computer getComputer();
6 }
```

Ln 6, Col 2 Spaces: 4 UTF-8 CRLF {} Java

This screenshot shows an IDE window with a file named 'task1&2'. The code defines a class 'GamingComputerBuilder' that implements the 'ComputerBuilder' interface. It includes a private 'computer' attribute and overrides methods to build a gaming computer with specific hardware.

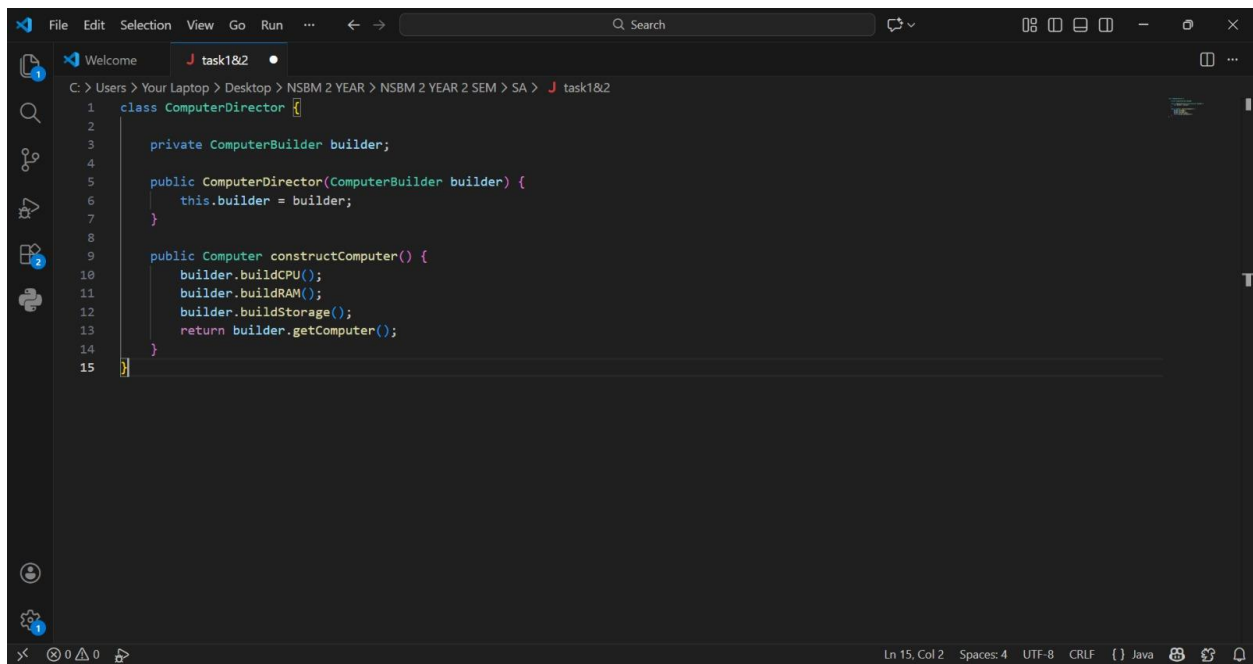
```
1 class GamingComputerBuilder implements ComputerBuilder {
2
3     private Computer computer = new Computer();
4
5     @Override
6     public void buildCPU() {
7         computer.setCpu("Intel Core i9");
8     }
9
10    @Override
11    public void buildRAM() {
12        computer.setRam(32);
13    }
14
15    @Override
16    public void buildStorage() {
17        computer.setStorage(2000);
18    }
19
20    @Override
21    public Computer getComputer() {
22        return computer;
23    }
24 }
```

The status bar at the bottom indicates the cursor is at line 24, column 2, with 4 spaces, UTF-8 encoding, and CRLF line endings.

This screenshot shows the same IDE window with the file 'task1&2'. The code defines a class 'NormalComputerBuilder' that implements the 'ComputerBuilder' interface. It includes a private 'computer' attribute and overrides methods to build a normal computer with specific hardware.

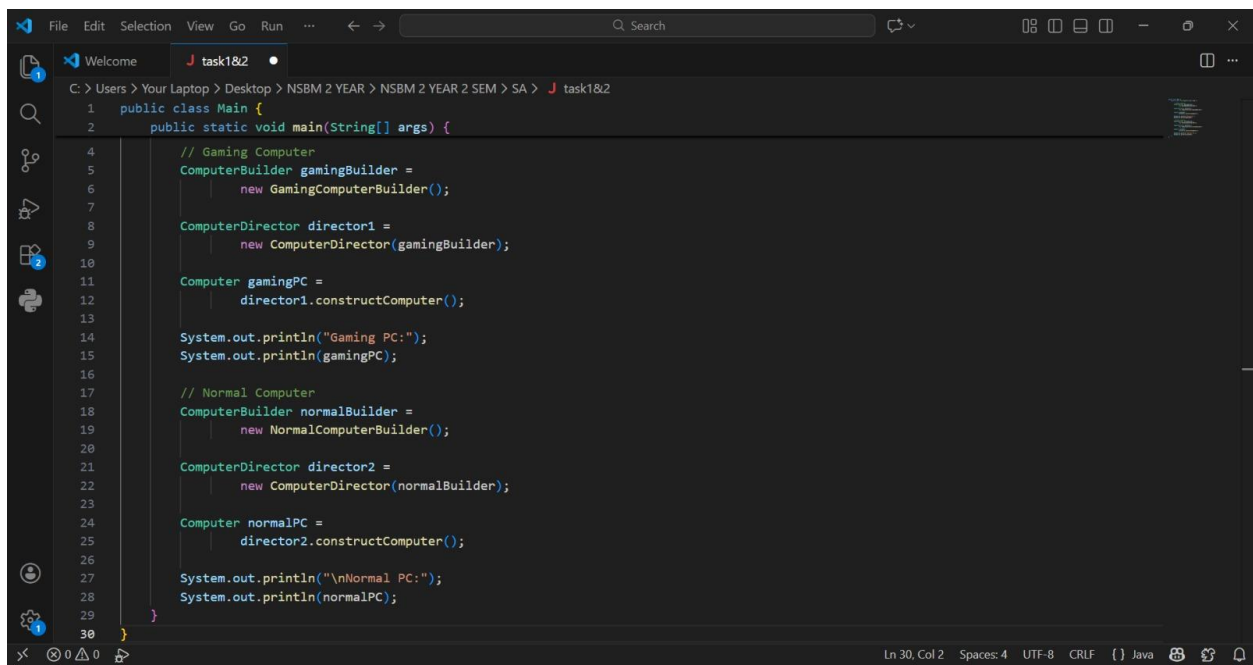
```
1 class NormalComputerBuilder implements ComputerBuilder {
2
3     private Computer computer = new Computer();
4
5     @Override
6     public void buildCPU() {
7         computer.setCpu("Intel Core i5");
8     }
9
10    @Override
11    public void buildRAM() {
12        computer.setRam(8);
13    }
14
15    @Override
16    public void buildStorage() {
17        computer.setStorage(512);
18    }
19
20    @Override
21    public Computer getComputer() {
22        return computer;
23    }
24 }
```

The status bar at the bottom indicates the cursor is at line 24, column 2, with 4 spaces, UTF-8 encoding, and CRLF line endings.



This screenshot shows an IDE window with a single file named 'task1&2'. The code defines a 'ComputerDirector' class. It has a private 'ComputerBuilder' field named 'builder'. The constructor 'ComputerDirector(ComputerBuilder builder)' initializes 'this.builder' to the passed 'builder'. The 'constructComputer()' method calls 'builder.buildCPU()', 'builder.buildRAM()', and 'builder.buildStorage()' in sequence, then returns 'builder.getComputer()'. The status bar at the bottom indicates the cursor is at line 15, column 2, with 4 spaces, UTF-8 encoding, and CRLF line endings.

```
1 class ComputerDirector {
2
3     private ComputerBuilder builder;
4
5     public ComputerDirector(ComputerBuilder builder) {
6         this.builder = builder;
7     }
8
9     public Computer constructComputer() {
10        builder.buildCPU();
11        builder.buildRAM();
12        builder.buildStorage();
13        return builder.getComputer();
14    }
15 }
```



This screenshot shows the same IDE window with the 'task1&2' file. The code defines a 'Main' class with a 'main' method. It creates a 'Gaming Computer' by instantiating 'GamingComputerBuilder', then 'ComputerDirector director1' with the builder, and finally 'Computer gamingPC' by calling 'director1.constructComputer()'. It prints 'Gaming PC:' and the 'gamingPC' object. Similarly, it creates a 'Normal Computer' by instantiating 'NormalComputerBuilder', then 'ComputerDirector director2', and finally 'Computer normalPC' by calling 'director2.constructComputer()'. It prints '\nNormal PC:' and the 'normalPC' object. The status bar at the bottom indicates the cursor is at line 30, column 2, with 4 spaces, UTF-8 encoding, and CRLF line endings.

```
1 public class Main {
2     public static void main(String[] args) {
3
4         // Gaming Computer
5         ComputerBuilder gamingBuilder =
6             new GamingComputerBuilder();
7
8         ComputerDirector director1 =
9             new ComputerDirector(gamingBuilder);
10
11         Computer gamingPC =
12             director1.constructComputer();
13
14         System.out.println("Gaming PC:");
15         System.out.println(gamingPC);
16
17         // Normal Computer
18         ComputerBuilder normalBuilder =
19             new NormalComputerBuilder();
20
21         ComputerDirector director2 =
22             new ComputerDirector(normalBuilder);
23
24         Computer normalPC =
25             director2.constructComputer();
26
27         System.out.println("\nNormal PC:");
28         System.out.println(normalPC);
29     }
30 }
```